

Programación Científica · UNAL · 2026-1

Integración Numérica

Módulo 4

Cuadratura · Regla del Trapecio · Regla de Simpson

**¿Cómo calculamos el área cuando no hay
fórmula?**

El problema

Queremos evaluar la integral definida:

$$I = \int_a^b f(x) dx$$

Sabemos por el Teorema Fundamental del Cálculo que si $F'(x) = f(x)$, entonces $I = F(b) - F(a)$.

El obstáculo

- ¿Qué pasa si $F(x)$ no se puede expresar con funciones elementales?
- ¿O si **solo tenemos datos experimentales** medidos en puntos discretos x_i ?

Necesitamos aproximar el valor de la integral sumando áreas geométricas.

Cuadratura Numérica

La idea central de la integración numérica (o cuadratura) es aproximar la integral como una **suma ponderada** de evaluaciones de la función:

$$\int_a^b f(x) dx \approx \sum_{i=0}^n w_i f(x_i)$$

- ▶ x_i : Nodos o puntos de evaluación.
- ▶ w_i : Pesos asociados a cada nodo.

Estrategia de Newton-Cotes

Dividimos el intervalo $[a, b]$ en n subintervalos de ancho $h = \frac{b-a}{n}$. Luego, interpolamos $f(x)$ usando polinomios de grado bajo (líneas, parábolas) en cada subintervalo e integramos ese polinomio de forma exacta.

Métodos de Newton-Cotes

La Regla del Trapecio

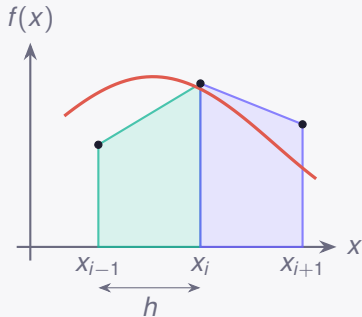
Aproximamos la función mediante una línea recta entre $f(x_i)$ y $f(x_{i+1})$. El área bajo esta línea es un trapecio.

Fórmula en un solo intervalo (tamaño h):

$$\int_{x_i}^{x_{i+1}} f(x) dx \approx \frac{h}{2} [f(x_i) + f(x_{i+1})]$$

Regla del Trapecio Compuesta para n intervalos:

$$I \approx \frac{h}{2} \left[f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n) \right]$$



El error global de la regla del trapecio escala como $\mathcal{O}(h^2)$.

La Regla de Simpson

En lugar de conectar los puntos con rectas, usamos grupos de **tres puntos** consecutivos (x_{i-1}, x_i, x_{i+1}) para ajustar una parábola (polinomio de grado 2).

Fórmula de Simpson 1/3 (Compuesta)

Para aplicarla, el número de subintervalos n **debe ser par**.

$$I \approx \frac{h}{3} \left[f(x_0) + 4 \sum_{i \text{ impar}} f(x_i) + 2 \sum_{i \text{ par}} f(x_i) + f(x_n) \right]$$

Ventajas:

- ▶ Gana mucha precisión con muy poco esfuerzo computacional extra.
- ▶ Su error global escala como $\mathcal{O}(h^4)$.



Integración en la práctica

Usando NumPy y SciPy

```
import numpy as np
import scipy.integrate as integrate

# 1. Definir los nodos y la funcion
x = np.linspace(0, np.pi, 11) # 10 paneles (11 puntos)
y = np.sin(x)

# 2. Regla del Trapecio (con NumPy)
I_trap = np.trapz(y, x)
print(f"Trapezio: {I_trap:.5f}") # Salida: 1.98352

# 3. Regla de Simpson (con SciPy)
I_simp = integrate.simpson(y, x)
print(f"Simpson: {I_simp:.5f}") # Salida: 2.00011
```

Cuadratura Gaussiana

¿Y si elegimos los puntos de forma inteligente?

El límite de Newton-Cotes

En Trapecio y Simpson, los puntos x_i **están fijos y equidistantes**. Solo jugamos con los pesos w_i .

La genialidad de Gauss

¿Qué pasa si tratamos tanto a los pesos w_i como a las posiciones de los nodos x_i como **incógnitas** y los optimizamos simultáneamente?

- ▶ Con n puntos equidistantes, integramos polinomios de grado $n - 1$ exactamente.
- ▶ Con n puntos de Gauss, integramos polinomios de grado $2n - 1$ exactamente. ¡El doble de precisión por el mismo costo computacional!

`scipy.integrate.quadrature`

Regla de Gauss-Legendre (2 puntos)

La fórmula estándar de cuadratura gaussiana está definida en el **intervalo canónico** $[-1, 1]$:

$$\int_{-1}^1 f(x) dx \approx \sum_{i=1}^n w_i f(x_i)$$

Para $n = 2$, los parámetros óptimos (derivados matemáticamente) son:

Punto i	Nodo x_i	Peso w_i
1	$-1/\sqrt{3} \approx -0.57735$	1
2	$1/\sqrt{3} \approx 0.57735$	1

Fórmula de 2 puntos

$$\int_{-1}^1 f(x) dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right)$$

Integración de Monte Carlo

El poder de la aleatoriedad en altas dimensiones

La maldición de la dimensionalidad

Gauss, Simpson y Trapecio son fantásticos en 1 dimensión. Pero, ¿qué pasa si queremos integrar en 10 dimensiones?

$$\int \cdots \int f(x_1, \dots, x_{10}) dx_1 \dots dx_{10}$$

Si usamos una malla de **apenas 10 puntos** por dimensión:

- ▶ $1D \rightarrow 10^1 = 10$ evaluaciones.
- ▶ $2D \rightarrow 10^2 = 100$ evaluaciones.
- ▶ $10D \rightarrow 10^{10} = 10.000.000.000$ evaluaciones.

El problema de los métodos de malla

El costo computacional crece exponencialmente con el número de dimensiones. A esto se le conoce como la **Maldición de la Dimensionalidad**.

La solución probabilística

El método de **Monte Carlo** abandona las mallas estructuradas. En su lugar, lanza "dardos" al azar en el dominio de integración.

La fórmula básica del valor esperado en 1D es:

$$\int_a^b f(x) dx \approx \frac{b-a}{N} \sum_{i=1}^N f(x_i)$$

Donde $x_1, x_2, \dots, x_N$ son números aleatorios generados de una distribución **uniforme** entre a y b .

La magia de Monte Carlo

El error del método de Monte Carlo disminuye a una tasa de $\mathcal{O}(1/\sqrt{N})$, **independientemente del número de dimensiones**. En dimensiones altas, Monte Carlo no es solo una alternativa, ¡es la única opción!

Ejercicio Práctico: Estimando π con Dardos

Dado un cuadrado de lado 2 centrado en el origen. Dentro de él hay un círculo de radio 1.

Si lanzamos N dardos aleatorios sobre el cuadrado, la probabilidad de caer dentro del círculo es:

$$P \approx \frac{\text{Área Círculo}}{\text{Área Cuadrado}} = \frac{\pi}{4}$$

Condición

Un dardo (x, y) está dentro del círculo si:

$$x^2 + y^2 \leq 1$$

En Python

1. Genere $N = 10^6$ valores de x aleatorios uniformes entre $[-1, 1]$.
2. Genere $N = 10^6$ valores de y independientes entre $[-1, 1]$.
3. Cuente cuántos puntos cumplen $x**2 + y**2 \leq 1$.
4. Multiplique esa fracción por el área del cuadrado (4).
5. Imprima el error respecto a `np.pi`.

El volumen de una Hiperesfera (10D)

En 2D, estimamos el área de un círculo ($x_1^2 + x_2^2 \leq 1$).

¿Qué pasa si queremos calcular el volumen de una **hiperesfera en 10 dimensiones**?

La Condición Analítica

Un punto $(x_1, x_2, \dots, x_{10})$ pertenece a la hiperesfera de radio 1 si:

$$\sum_{i=1}^{10} x_i^2 \leq 1$$

El fracaso de los métodos clásicos

Si usamos la Regla de Simpson con una malla muy pobre de **apenas 10 puntos** por eje:

- ▶ En 2D: $10^2 = 100$ puntos.
- ▶ En 10D: $10^{10} = 10.000.000.000$ evaluaciones.

Recapitulando

El Problema de Integrar

Aproximar el área sumando áreas geométricas conocidas

Trapecio vs. Simpson

Interpolación lineal vs. cuadrática: $\mathcal{O}(h^2)$ vs $\mathcal{O}(h^4)$

Cuadratura de Gauss

Puntos y pesos optimizados: el doble de precisión analítica

Monte Carlo

Integración estocástica: invencible en dimensiones altas

`mbastidaso@unal.edu.co`